The same security precautions have to be taken for Ajax actions as for "normal" ones. There is at least one exception, however: The output has to be escaped in the controller already, if the action doesn't render a view.

If you use the in_place_editor plugin, or actions that return a string, rather than rendering a view, *you have to escape the return value in the action*. Otherwise, if the return value contains a XSS string, the malicious code will be executed upon return to the browser. Escape any input value using the h() method.

7.7 Command Line Injection

Use user-supplied command line parameters with caution.

If your application has to execute commands in the underlying operating system, there are several methods in Ruby: exec(command), syscall(command), system(command) and `command`. You will have to be especially careful with these functions if the user may enter the whole command, or a part of it. This is because in most shells, you can execute another command at the end of the first one, concatenating them with a semicolon (;) or a vertical bar (|).
A countermeasure is to *use the `system(command, parameters)` method which passes command line parameters safely*.

```
system("/bin/echo","hello; rm *")
# prints "hello; rm *" and does not delete files
```

7.8 Header Injection

HTTP headers are dynamically generated and under certain circumstances user input may be injected. This can lead to false redirection, XSS or HTTP response splitting.

HTTP request headers have a Referer, User-Agent (client software), and Cookie field, among others. Response headers for example have a status code, Cookie and Location (redirection target URL) field. All of them are user-supplied and may be manipulated with more or less effort. *Remember to escape these header fields, too.* For example when you display the user agent in an administration area.
Besides that, it is *important to know what you are doing when building response headers partly based on user input.* For example you want to redirect the user back to a specific page. To do that you introduced a "referer" field in a form to redirect to the given address:

```
redirect_to params[:referer]
```

What happens is that Rails puts the string into the Location header field and sends a 302 (redirect) status to the browser. The first thing a malicious user would do, is this:

```
http://www.yourapplication.com/controller/action?referer=http://www.malicious.tld
```

And due to a bug in (Ruby and) Rails up to version 2.1.2 (excluding it), a hacker may inject arbitrary header fields; for example like this:

```
http://www.yourapplication.com/controller/action?referer=http://www.malicious.tld%
0d%0aX-Header:+Hi!
http://www.yourapplication.com/controller/action?referer=path/at/your/app%0d%0aLoc
ation:+http://www.malicious.tld
```

Note that "%0d%0a" is URL-encoded for "\r\n" which is a carriage-return and line-feed (CRLF) in Ruby. So the resulting HTTP header for the second example will be the following because the second Location header field overwrites the first.

```
HTTP/1.1 302 Moved Temporarily
(...)
Location: http://www.malicious.tld
```